MetaEvolve: Meta-Evolving Automated Discovery with Epistemic Memory

Anonymous ACL submission

Abstract

Large language model (LLM)-driven discovery has moved from evolving candidate programs to adapting search strategies, but most systems still leave a deeper layer fixed: how the system reads prior attempts, intervenes through the model, and converts outcomes into reusable methodological knowledge. We introduce MetaEvolve, a framework for meta-evolving the methods of discovery themselves. MetaEvolve represents each search move as a *discovery operator*, an executable unit that prepares evidence, generates a candidate with an LLM, and reflects on the evaluated outcome. These operators interact with a **hierarchical epistemic memory** over solutions, typed experiences, executable operators, and operator-design meta-experiences, while a **dual-loop evolution** process jointly improves candidate solutions (inner loop) and the operators that produce them (outer loop). Across eight mathematical optimization tasks, MetaEvolve achieves the best or tied-best open-framework result, and matches or improves AlphaEvolve-level results on Hexagon Packing-12, Erdős Minimum Overlap, and Third Autocorrelation. Codes are available at https://anonymous.4open.science/r/metaevolve.

1 Introduction

Large language model (LLM)-driven discovery is moving from one-shot generation toward long-horizon search (Novikov et al., 2025a; Sharma, 2025; Yan et al., 2026). In mathematical discovery, program synthesis, and systems optimization, the strongest systems now maintain populations of candidate programs, evaluate them with external objectives, and reuse prior attempts to guide future generations (Romera-Paredes et al., 2024a; Novikov et al., 2025b; Sharma, 2025; Lange et al., 2025). This shift raises a deeper question than how to sample the next candidate: *which parts of the discovery process itself should be learned?*

The field has so far progressed through two recognizable stages. The first stage is ✤ **candidate evolution**. Systems such as FunSearch (Romera-Paredes et al., 2024a), AlphaEvolve (Novikov et al., 2025a), and OpenEvolve (Sharma, 2025) automate a Baconian experimental loop (Bacon, 1620): propose artifacts, test them, preserve useful variants, and repeat. This has produced striking results, but the procedure around the loop is largely *fixed by hand*. The archive structure, parent selection rule, prompt template, feedback format, and reuse policy are specified before the run begins (Assumpção et al., 2025; Lange et al., 2025). In this stage, the system learns better answers, while the method for producing those answers remains mostly human-designed (Liu et al., 2026).

The second stage is ✤ **strategy evolution**. Recent systems begin to adapt how search itself is conducted: AdaEvolve (Cemri et al., 2026) coordinates exploration, resource allocation, and meta-guidance through improvement signals, while EvoX (Liu et al., 2026) directly evolves policies that select and vary prior candidates. This is a meaningful step upward. The system no longer asks only "*what candidate should be tested next?*" It also asks "*which search action should be taken next?*" Yet the learned object is still typically a search controller: a sampling rule, a variation policy, a schedule, or a lightweight composition of predefined choices.

We argue that LLM-driven discovery is ready for a third stage: ✤ **methodological evolution**. Scientific and mathematical progress often depends on objects larger than a candidate solution and richer than a sampling policy. Two attempts at the same problem may differ not by a local edit, but by a problem-solving stance. One solver introduces coordinates; another draws an auxiliary construction; another searches for an invariant, changes representation, relaxes the objective, or combines partial structures that were previously treated as incom-

patible. These are not merely *candidates*, because they describe families of future attempts. They are not merely *strategies*, because they specify what evidence matters, what kind of intervention is natural, and what lesson should be retained when the attempt succeeds or fails.

Current LLM evolution systems have little machinery for representing and improving such *methodological knowledge*. They may record that a program improved fitness, or that a policy obtained a higher reward, but the feedback is usually compressed into scalar progress or unstructured execution traces. As a result, the system can optimize within a discovery procedure, and in some cases adapt the procedure's control policy, but it cannot easily invent *new ways of seeing and attacking a problem* from its own accumulated experience. The missing unit is not just a better memory buffer; it is an evolvable representation of the problem-solving method itself.

Research question. Can automated discovery systems evolve its own problem-solving methods, more specifically, how it reads prior attempts, acts on a problem, and turns outcomes into reusable methodological knowledge?

We introduce `MetaEvolve`, a framework for *epistemic evolution*: the joint evolution of solutions, discovery methods, and the memories that make those methods reusable. The central abstraction in `MetaEvolve` is the **discovery operator**. A discovery operator is not a mutation template or a parent selector. It is an executable methodological unit: a procedure that decides what evidence to read, how to transform that evidence into an LLM intervention, and what semantic lesson to write back after the outcome is observed. In other words, `MetaEvolve` treats a search operator as the computational analogue of a problem-solving style.

`MetaEvolve` realizes this idea through a **Hierarchical Epistemic Memory** with four coupled objects: ❶ *solution lineages* with evaluations and operator provenance; ❷ *typed experiences* that verbalize errors, improvements, regressions, stagnation, and strategy observations; ❸ *executable operators* with performance histories; and ❹ *meta-experiences* that summarize reusable methodological patterns. The **inner loop** applies operators to produce solutions and write experiences; the **outer loop** reads these memories to synthesize new operators. Thus, while prior strategy-evolution methods adapt how candidates are selected and varied, `MetaEvolve` evolves the broader epistemic machinery that determines how a discovery system observes, intervenes, remembers, and changes its method over time. Our contributions are:

- We formulate *methodological evolution* as a third stage of LLM-driven discovery, moving beyond evolving candidates or search policies toward evolving reusable problem-solving methods.
- We introduce `MetaEvolve`, where executable discovery operators interact with a hierarchical epistemic memory through coupled inner solution evolution and outer operator evolution.
- We evaluate `MetaEvolve` on eight optimization tasks, showing strong open-framework performance and ablation evidence for operator evolution and epistemic memory.

2 Related Work

LLM-Driven Scientific Discovery LLM-driven discovery has moved from candidate evolution toward adaptive search. Early systems use evaluator-guided loops that generate, score, archive, and reuse candidate programs (Romera-Paredes et al., 2024a,b; Novikov et al., 2025a,b); open variants add islands, diversity mechanisms, pruning, and progress-aware reuse (Sharma, 2025; Mouret and Clune, 2015; Agrawal et al., 2025; Lange et al., 2025; Assumpção et al., 2025; Yan et al., 2026). More recent systems adapt parent selection, resource allocation, meta-guidance, or search policies from observed progress (Lange et al., 2025; Cemri et al., 2026; Liu et al., 2026), while agentic variants expose retrieval, proposal, evaluation, and memory update to model control (Qu et al., 2026; Lu et al., 2024; Yamada et al., 2025). `MetaEvolve` follows this trajectory but changes the unit of adaptation to executable discovery operators.

Meta Learning and Evolution. Meta-learning and adaptive evolution make optimization procedures learnable. Learning-to-optimize, symbolic optimizer discovery, and adaptive operator selection learn update rules, algorithmic forms, or online operator credit (Andrychowicz et al., 2016; Metz et al., 2019; Chen et al., 2022, 2023; Fialho, 2010). In LLM systems, related ideas appear as prompt evolution, reflective prompt optimization, compound-pipeline optimization, and memory-based reflection (Fernando et al., 2023; Guo et al., 2023; Agrawal et al., 2025; Khattab et al., 2023; Shinn et al., 2024; Madaan et al., 2023;

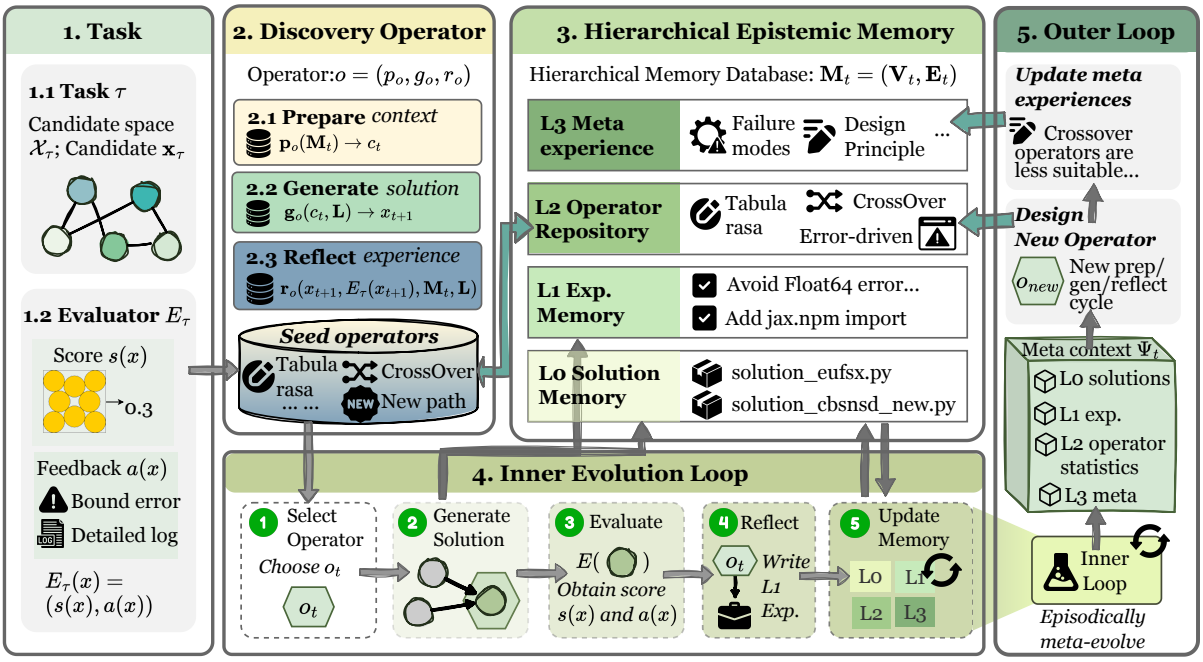


Figure 1: **MetaEvolve Overview**: the inner loop applies discovery operators to generate candidates, while the outer loop uses accumulated evidence to synthesize stronger ones.

Zhang et al., 2025; Packer et al., 2023). These works usually adapt one layer, such as an update rule, prompt, operator choice. MetaEvolve instead evolves a hierarchy of full discovery operators using both scalar progress and semantic experience.

3 Methodology

3.1 Problem Formulation

We consider evaluator-guided discovery for a task τ, with candidate space $\mathbf{X}_\tau$. Each candidate $x \in \mathbf{X}_\tau$ is scored by an external evaluator:

$$E_\tau : \mathbf{X}_\tau \to \mathbb{R} \times \mathbf{A}, \; E_\tau(x) = (s(x), a(x)), \quad (1)$$

where $s(x)$ is the scalar objective and $a(x)$ denotes auxiliary feedback such as traces, errors, tests, or task metrics. Under T evaluator calls, the goal is:

$$x_T^\star \in \arg\max_{x_i \in \mathbf{S}_T} s(x_i). \quad (2)$$

where $\mathbf{S}_T = \{x_i\}_{i=1}^T$ is the set of evaluated candidates. The key design choice is how x_{t+1} is constructed from history. We capture this choice with a **discovery operator**. At step t, the system maintains epistemic memory $\mathbf{M}_t$ and operators $\mathbf{O}_t \subset \mathbf{\Omega}_\tau$, where $\mathbf{\Omega}_\tau$ is the task-conditioned operator space. Each $\mathbf{o} \in \mathbf{\Omega}_\tau$ is an executable procedure:

$$\begin{aligned} \mathbf{o} &= (\mathbf{p}_o, \mathbf{g}_o, \mathbf{r}_o), \\ \mathbf{p}_o(\mathbf{M}_t) &\to c_t, \; \mathbf{g}_o(c_t, \mathbf{L}) \to x_{t+1}, \\ \mathbf{r}_o(x_{t+1}, E_\tau(x_{t+1}), \mathbf{M}_t, \mathbf{L}) &\to z_{t+1}, \end{aligned} \quad (3)$$

where $\mathbf{p}_o$ prepares context c_t, $\mathbf{g}_o$ uses the LLM $\mathbf{L}$ to produce x_{t+1}, and $\mathbf{r}_o$ converts the evaluated outcome into structured experience z_{t+1}. Thus an operator specifies a complete epistemic action: what to read, how to intervene, and what to write back. Many existing systems instantiate restricted regions of $\mathbf{\Omega}_\tau$: fixed evolution frameworks hand-design $\mathbf{p}_o, \mathbf{g}_o$ and reduce $\mathbf{r}_o$ to archive update, while adaptive systems modify selection, resource use, or search schedules from progress signals. MetaEvolve instead makes $\mathbf{o}$ explicit, persistent, executable, reflective, and itself subject to meta-evolution. The search process therefore optimizes over both candidates and the procedures that generate them:

$$\begin{aligned} \mathbf{M}_{t+1} &= \mathbf{M}_t \oplus \{(\mathbf{o}_t, \; x_{t+1}, \; y_{t+1}, \; z_{t+1})\}, \\ y_{t+1} &= E_\tau(x_{t+1}), \end{aligned} \quad (4)$$

where $\oplus$ denotes insertion into the hierarchical memory graph and $\mathbf{o}_t \in \mathbf{O}_t$ is the operator used at step t. The problem is therefore to improve both high-scoring candidates and the operators that make future search more informed.

3.2 Hierarchical Epistemic Memory

The operator view requires memory richer than a flat solution archive. A discovery system must remember not only *what* was tried, but also *why* it worked or failed and *which method* produced it. We therefore represent memory as:

$$\mathbf{M}_t = (\mathbf{V}_t, \mathbf{E}_t), \; \mathbf{V}_t = \mathbf{V}_t^x \cup \mathbf{V}_t^z \cup \mathbf{V}_t^o \cup \mathbf{V}_t^m, \quad (5)$$

where the four node sets correspond to *solutions*, *experiences*, *operators*, and *meta-experiences*. Edges in $\mathbf{E}_t$ record lineage, provenance, reflection, derivation, and guidance.

L0: Solution memory. Each solution node stores the evaluated solution together with its context:

$$\mathbf{v}_i^x = (x_i, s_i, a_i, \mathbf{o}_i, t_i) \in \mathbf{V}_t^x, \tag{6}$$

where $\mathbf{o}_i$ is the operator that generated x_i. L0 has intra-layer lineage edges $x_j \to x_i$ when x_i is produced using x_j as a parent, reference, or repair target. These edges support standard evolutionary queries such as top solutions, diverse solutions, ancestor chains, and recent score deltas.

L1: Experience memory. Raw evaluator feedback is often too low-level for future operators. After evaluation, $\mathbf{r}_o$ writes a structured experience node, denoted as:

$$\mathbf{v}_i^z = (\kappa_i,\ m_i,\ \Delta s_i,\ \mathbf{o}_i,\ t_i) \in \mathbf{V}_t^z, \tag{7}$$

where κ_i is the experience type, m_i is a natural-language summary, and Δs_i records the associated score change. We use five types: `error_pattern`, `improvement_insight`, `regression_warning`, `stagnation_note`, and `strategy_observation`. L1 connects downward to L0 in two directions: $x_i \to z_i$ when an evaluated solution produces an experience, and $z_j \to x_i$ when an experience is retrieved by $\mathbf{p}_o$ and used to generate a later solution. Thus L1 is the semantic bridge between scalar evaluation and future search.

L2: Operator memory. Operators are first-class memory objects rather than transient prompts:

$$\mathbf{v}_j^o = (\mathrm{code}(\mathbf{o}_j),\ n_j,\ \bar{r}_j,\ \rho_j,\ \boldsymbol{\psi}_j) \in \mathbf{V}_t^o, \tag{8}$$

where n_j is the number of deployments, $\bar{r}_j$ is mean reward, ρ_j is success rate, and $\boldsymbol{\psi}_j$ is an API-usage profile describing which memory queries the operator uses. L2 links downward through provenance edges $\mathbf{o}_j \to x_i$ and $\mathbf{o}_j \to z_i$ when an operator generates a solution or writes an experience. It also has intra-layer derivation edges $\mathbf{o}_a \to \mathbf{o}_b$ when a new operator is synthesized from, mutates, or explicitly reads an earlier operator.

L3: Meta-experience memory. The top layer abstracts from individual operators to methodological patterns. A meta-experience node is a super-node over a set of operators:

$$\mathbf{v}_k^m = (\mathbf{O}_k^m,\ u_k,\ \eta_k) \in \mathbf{V}_t^m,\ \mathbf{O}_k^m \subseteq \mathbf{V}_t^o, \tag{9}$$

where $\mathbf{O}_k^m$ is the represented operator set, u_k is an agent-generated methodological summary, and η_k stores aggregate evidence such as typical use cases or failure modes. L3 links downward as $\mathbf{v}_k^m \to \mathbf{o}_j$ for member operators and $\mathbf{v}_k^m \to \mathbf{o}_{new}$ when its summary guides the outer loop to synthesize a new operator. Thus $\mathbf{M}_t$ gives the outer loop not merely a leaderboard of operators, but a structured vocabulary of reusable discovery methods.

3.3 Dual-Loop Meta Evolution

With discovery operators and hierarchical memory defined, we can describe `MetaEvolve` as two coupled evolutionary processes. The **inner loop** evolves solutions under the current operator population $\mathbf{O}_t$; the **outer loop** evolves $\mathbf{O}_t$ itself using the evidence accumulated in $\mathbf{M}_t$. Thus each evaluated candidate is both an attempt at solving τ and evidence about the method that produced it.

Inner loop: reflective solution evolution. At each step, `MetaEvolve` selects an operator from L2 using its empirical utility and uncertainty:

$$\mathbf{o}_t = \arg\max_{\mathbf{o}_j \in \mathbf{O}_t} \bar{r}_j + C\sqrt{\frac{\ln(1+N_t)}{1+n_j}}, \tag{10}$$

where n_j and $\bar{r}_j$ are stored in $\mathbf{v}_j^o$, $N_t = \sum_j n_j$, and $C = \sqrt{2}$ in our implementation. The selected operator then performs one complete epistemic action:

$$\begin{gathered} c_t = \mathbf{p}_{o_t}(\mathbf{M}_t),\ x_{t+1} = \mathbf{g}_{o_t}(c_t, \mathbf{L}), \\ y_{t+1} = E_\tau(x_{t+1}),\ z_{t+1} = \mathbf{r}_{o_t}(x_{t+1}, y_{t+1}, \mathbf{M}_t, \mathbf{L}). \end{gathered} \tag{11}$$

The resulting tuple $(\mathbf{o}_t, x_{t+1}, y_{t+1}, z_{t+1})$ updates L0, L1, and L2 simultaneously: x_{t+1} becomes a solution node, z_{t+1} becomes an experience node, and $\mathbf{o}_t$ receives new credit and provenance edges. We use a task-normalized reward $r_t = \mathrm{Norm}(s(x_{t+1}) - s_{\mathrm{ref}})$, where s_{ref} is the parent or incumbent score depending on the operator.

Outer loop: operator meta-evolution. The outer loop is triggered when enough evidence has accumulated or when recent progress stalls. Let $\Delta_W = \max_{i\in[t-W,t]} s(x_i) - \max_{i<t-W} s(x_i)$. We write the trigger compactly as

$$\Gamma_t = \begin{cases} 1, & (t \bmod T_{\mathrm{meta}} = 0) \vee (\Delta_W \le \epsilon), \\ 0, & \text{otherwise}, \end{cases} \tag{12}$$

where T_{meta} is the meta-evolution period, W is the stagnation window, and ϵ is a task-level progress threshold. When $\Gamma_t = 1$, `MetaEvolve` builds a

Table 1: **Main results on mathematical optimization tasks.** We report mean and best scores over three runs; ↑/↓ denotes higher/lower is better. All open frameworks use the same backbone model and a 100-evaluation budget. In the best-score rows, **bold** marks the best open-framework result within each model group; bold in reference columns denotes the strongest published reference. Light green cells mark the global best performance in each best-score row across all reported columns. For rounded ties, we compare the underlying values: on heilbronn_convex_14, the rounded 0.0278 results are slightly below AlphaEvolve ($0.0278037352 < 0.0278355$); on hexagon_packing_12, `MetaEvolve` reaches 0.2537010917, slightly above AlphaEvolve 0.25368 and EvoX 0.2536995734; on erdos_min_overlap, `MetaEvolve` reaches 0.3809020817, better than AlphaEvolve 0.3809230351 and the Shinka/EvoX values $0.380950169/0.3809334338$.

				DeepSeek-V4-Pro				Gemini-3.1-Pro			
Task	**Stat**	**Human**	**AlphaEvolve**	**OpenEvolve**	**Shinka**	**EvoX**	**`MetaEvolve`**	**OpenEvolve**	**Shinka**	**EvoX**	**`MetaEvolve`**
Circle Packing Rect (↑)	Mean	–	–	2.2715	2.3169	2.3605	2.3626	2.3651	2.3658	2.3658	2.3658
	Best	2.3640	2.3658	2.3616	2.3530	2.3608	**2.3634**	**2.3658**	**2.3658**	**2.3658**	**2.3658**
heilbronn_convex_13 (↑)	Mean	–	–	0.0256	0.0296	0.0298	0.0302	0.0283	0.0295	0.0298	0.0306
	Best	0.0306	0.0309	0.0263	0.0298	0.0302	**0.0303**	0.0279	0.0295	0.0299	**0.0308**
heilbronn_convex_14 (↑)	Mean	–	–	0.0239	0.0246	0.0248	0.0255	0.0267	0.0274	0.0277	0.0278
	Best	–	0.0278	0.0241	0.0251	0.0257	**0.0259**	0.0269	**0.0278**	0.0277	**0.0278**
hexagon_packing_11 (↑)	Mean	–	–	0.2095	0.2415	0.2478	0.2541	0.2538	0.2537	0.2532	0.2543
	Best	–	0.2545	0.2536	0.2498	0.2541	**0.2545**	0.2540	0.2541	0.2541	**0.2545**
hexagon_packing_12 (↑)	Mean	–	–	0.2077	0.2492	0.2519	0.2524	0.2518	0.2091	0.2524	0.2519
	Best	–	0.2537	0.2484	0.2536	0.2534	**0.2537**	0.2537	0.2536	0.2537	**0.2537**
erdos_min_overlap (↓)	Mean	–	–	0.3817	0.3816	0.3812	0.3809	0.3812	0.3809	0.3809	0.3809
	Best	-	0.3809	0.3811	0.3812	0.3810	**0.3809**	0.3809	0.3810	0.3809	**0.3809**
third_autocorr_ineq (↓)	Mean	–	–	1.4696	1.4619	1.4610	1.4595	1.4593	1.4614	1.4591	1.4575
	Best	1.4581	1.4557	1.4680	1.4619	**1.4560**	1.4587	1.4564	1.4587	1.4560	**1.4551**
first_autocorr_ineq (↓)	Mean	–	–	1.5185	1.5184	1.5091	1.5101	1.5098	1.5077	1.5056	1.5050
	Best	–	1.5053	1.5128	1.5133	**1.5073**	1.5083	1.5073	1.5073	1.5054	**1.5053**

Algorithm 1: Workflow of `MetaEvolve`.

Input: Task τ; evaluator E_τ; LLM $\mathbf{L}$; initial memory $\mathbf{M}_0$; seed operators $\mathbf{O}_0$; budget T
Output: Best candidate $x_T^\star$ and operator set $\mathbf{O}_T$

for $t = 0, \dots, T-1$ **do**
  Select $\mathbf{o}_t \in \mathbf{O}_t$ by UCB over L2 statistics
  $c_t \leftarrow \mathbf{p}_{o_t}(\mathbf{M}_t)$; $x_{t+1} \leftarrow \mathbf{g}_{o_t}(c_t, \mathbf{L})$
  $y_{t+1} \leftarrow E_\tau(x_{t+1})$; $z_{t+1} \leftarrow \mathbf{r}_{o_t}(x_{t+1}, y_{t+1}, \mathbf{M}_t, \mathbf{L})$
  Update $\mathbf{M}_{t+1}$ with $(\mathbf{o}_t, x_{t+1}, y_{t+1}, z_{t+1})$ and refresh operator statistics
  if $t \bmod T_{\text{meta}} = 0$ *or recent progress stalls* **then**
    Build Ψ_t from L0–L3 memory evidence
    Generate $\mathbf{o}_{new} \sim \mathbf{G}_{op}(\cdot \mid \Psi_t)$
    Validate $\mathbf{o}_{new}$ for at most three attempts; register it in L2 if valid
    Prune low-utility or redundant operators when needed
return $x_T^\star = \arg\max_{x_i \in \mathbf{S}_T} s(x_i)$ and $\mathbf{O}_T$

meta-context Ψ_t from the four memory layers: recent score deltas and lineage transitions from L0, typed experience summaries from L1, operator statistics and source code from L2, and methodological summaries from L3. A generator $\mathbf{G}_{op}$ then proposes a new executable operator:

$$\mathbf{o}_{new} \coloneqq (\mathbf{p}_{new}, \mathbf{g}_{new}, \mathbf{r}_{new}) \sim \mathbf{G}_{op}(\cdot \mid \Psi_t), \qquad (13)$$

Crucially, the generated object is not a natural-language hint but source code implementing the same interface as Eq. 3. It may change what memory is read, how the LLM is prompted, and what experience is written back; representative prompts are listed in Section E.

Validation and registration. Because operators execute inside the search loop, `MetaEvolve` validates each $\mathbf{o}_{new}$ against the required interface before deployment. Generation is retried at most three times; if all attempts fail, no operator is added and the current $\mathbf{O}_t$ is preserved. If validation succeeds, $\mathbf{o}_{new}$ is inserted into L2 with derivation edges to the evidence used in Ψ_t. Low-value or redundant operators may later exit the population, allowing the system to accumulate a compact library of useful operators that can be reused within a task or transferred across tasks.

The two loops, therefore, close the epistemic evolution cycle: solutions produce experiences, experiences inform operators, operators form meta-experiences, and meta-experiences guide the invention of future discovery operators. The overall algorithm workflow is presented in Algo. 1.

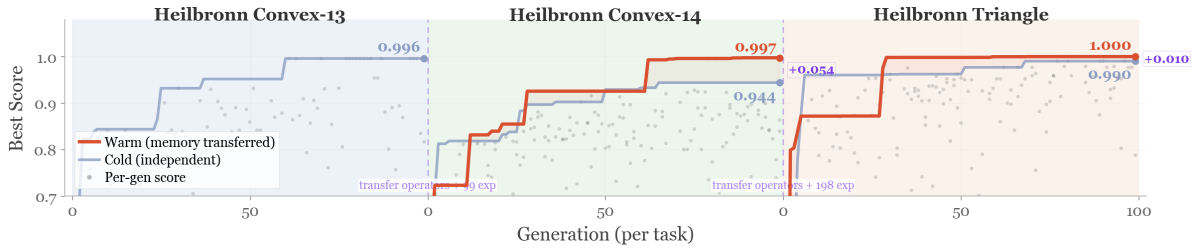


Figure 2: **Continual discovery across a Heilbronn task chain.** We run three related tasks sequentially (Convex-13 → Convex-14 → Triangle, 100 evaluations each). The *warm* condition transfers all three memory layers (L1 experiences, L2 operators, L3 meta-state) from each completed task to the next; the *cold* baseline runs each task independently from scratch. Scattered dots show individual per-generation scores.

4 Experiments

4.1 Experiment Setup

Tasks. We evaluate `MetaEvolve` on evaluator-guided program discovery tasks where each candidate is an executable solution and the only objective signal comes from a task-specific evaluator E_τ. Following EvoX (Liu et al., 2026), the reported suite spans geometric packing, extremal point configurations, overlap minimization, autocorrelation inequalities, and signal-processing design; task details are in Section B.

Baselines and models. We compare `MetaEvolve` against OpenEvolve (Sharma, 2025), ShinkaEvolve (Lange et al., 2025) and EvoX (Liu et al., 2026). For tasks with published references, we also report human-best and AlphaEvolve (Novikov et al., 2025a) results when available. All open-source methods use the same task specification, seed program when applicable, evaluator, backbone model, and total evaluation budget. We evaluate each open framework with DEEPSEEK-V4-PRO and GEMINI-3.1-PRO.

Budget and evaluation. Unless otherwise stated, each open method receives $T = 100$ evaluator calls per task. We run three independent trials and report both mean and best score; mean reflects robustness across stochastic runs, while best reflects the frontier reached by each framework. For `MetaEvolve`, $\mathbf{O}_0$ contains 6 seed discovery operators covering incremental mutation, crossover, diversity preservation, and tabula-rasa exploration (see Sections A and G). Operator selection uses Equation (10) with $C = \sqrt{2}$. The outer loop is checked every $T_{\text{meta}} = 20$ evaluations and whenever a window of $W = 10$ evaluations improves by at most $\epsilon = 10^{-4}$. We cap the active operator population at 16.

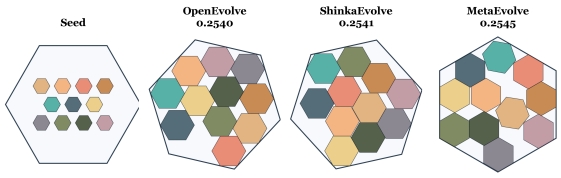


Figure 3: Visualization of the solutions discovered by OpenEvolve, ShinkaEvolve and MetaEvolve on hexagon_packing_11 problem.

4.2 Main Results

Table 1 reports the main comparison under the matched evaluation budget. `MetaEvolve` is strongest or tied strongest on the main reported tasks, especially under GEMINI-3.1-PRO: it matches the best packing baselines on Circle Packing Rect and Hexagon Packing-11 (2.3658/0.2545), slightly improves Hexagon Packing-12 over EvoX and AlphaEvolve (0.2537010917 vs. 0.2536995734/0.25368), improves over EvoX on Heilbronn Convex-13 (0.0308 vs. 0.0299), and surpasses AlphaEvolve on Erdős Minimum Overlap (0.3809020817 vs. 0.3809230351) and Third Autocorrelation (1.4551 vs. 1.4557, lower is better). Under DEEPSEEK-V4-PRO, `MetaEvolve` also leads the open baselines on core packing, geometry, and overlap tasks, including Heilbronn Convex-13/14 (0.0303/0.0259) and Hexagon Packing-11/12 (0.2545/0.2537). The qualitative examples in Figure 3 and Figure 9 show two concrete solutions discovered by `MetaEvolve`, with additional visualization and program sketches in Sections C and F.

4.3 Continual Discovery

Because the hierarchical memory is decoupled from any single task, it can be transferred to bootstrap search on a related problem. We test this on a chain of three Heilbronn geometry tasks of increasing difficulty (Convex-13 → Convex-14 → Triangle), transferring L1-L3 memory after each

Table 2: **Ablation of hierarchical epistemic memory.** Each task reports the absolute benchmark metric and the normalized relative score in parentheses. ↑ / ↓ denotes higher/lower is better.

Configuration	heilbronn convex 13 min area ↑ / rel. ↑	third autocorr C_3 ↓ / rel. ↑
`MetaEvolve`	0.0308 (0.9957)	1.4551 (1.0003)
w/o L1 (experience)	0.03021 (0.9767)	1.4575 (0.9987)
w/o L2 (operator evolution)	0.02949 (0.9533)	1.4565 (0.9994)
w/o L3 (meta-experience)	0.03003 (0.9708)	1.4584 (0.9981)

task completes (Figure 2). With 100 evaluations per task, warm-start runs outperform cold-start baselines on every stage: $+0.054$ on Convex-14 and $+0.010$ on Triangle, with the warm Triangle reaching 0.999 versus 0.990 for the cold run. Notably, the warm-start Triangle surpasses the cold-start final score within 30 evaluations—less than a third of the full budget—indicating that transferred operators and experiences provide a substantial head start rather than merely a marginal improvement.

4.4 Ablation Study

To isolate the contribution of each memory layer, we disable one layer at a time on *heilbronn_convex_13* and *third_autocorr_ineq* (Table 2). On the geometric task, removing L2 operator evolution causes the largest degradation, reducing relative score from 0.9957 to 0.9533, while removing L3 and L1 also lowers performance to 0.9708 and 0.9767, respectively. On *third_autocorr_ineq*, all ablations remain close to the full system but still fall below it ($1.0003 \rightarrow$ 0.9981–0.9994), suggesting that this smoother analytic task is less sensitive to any single memory layer. Overall, the ablation supports the intended hierarchy: L2 provides direct adaptive search capacity, L3 contributes longer-horizon methodological coordination, and L1 supplies typed evidence that makes operator synthesis more targeted.

4.5 Sensitivity Analysis

The outer-loop interval T_{meta} controls how frequently the system synthesizes new operators. Too frequent meta-evolution wastes evaluations on poorly grounded operators; too infrequent delays adaptation. We vary $T_{\text{meta}} \in \{3, 5, 10, 20\}$ on *hexagon_packing_12* with Gemini-3.1-Pro, keeping all other hyperparameters fixed. As shown in Figure 4, $T_{\text{meta}} = 10$ and $T_{\text{meta}} = 20$ both exceed the AlphaEvolve reference, while $T_{\text{meta}} = 3$ and $T_{\text{meta}} = 5$ fall slightly short. This is consistent with the outer loop needing a minimum evidence

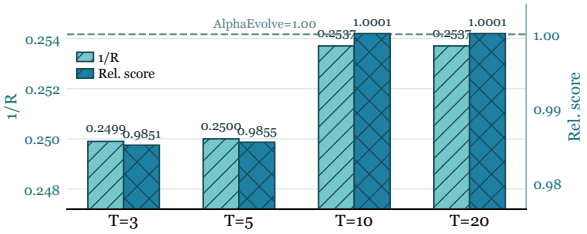


Figure 4: **Sensitivity to T_{meta}.** Varying the outer-loop interval on hexagon_packing_12 (Gemini-3.1-Pro). Bars show the absolute metric $1/R$ and normalized relative score; the dashed line marks AlphaEvolve at 1.00.

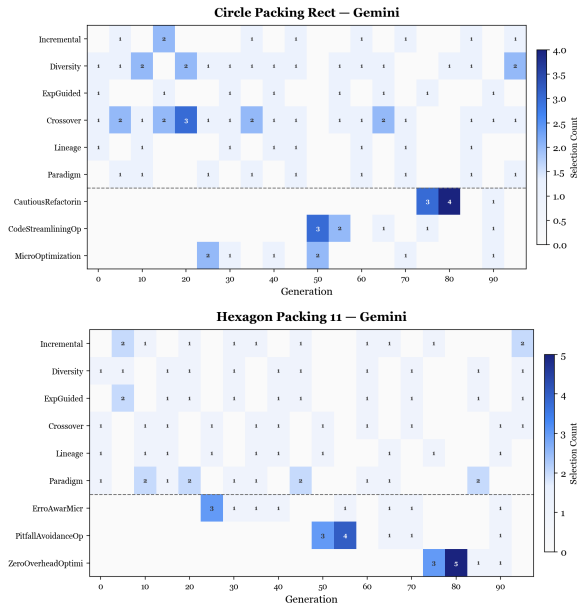


Figure 5: **Operator selection frequency** on two representative tasks. Top: circle_packing_rect; bottom: hexagon_packing_11. Rows are operators and columns are generation bins; numbers indicate selection count per bin. Dynamic operators appear after outer-loop updates and progressively absorb selections from the seed operators, with stronger concentration on hexagon_packing_11.

window to synthesize well-targeted operators. The small performance gap between $T_{\text{meta}} = 10$ and $T_{\text{meta}} = 20$ indicates that the system is robust to moderate changes in meta-evolution frequency. We use $T_{\text{meta}} = 20$ throughout the main experiments as it provides a good trade-off between adaptation speed and evidence quality.

4.6 Framework Analysis

We examine the internal dynamics of `MetaEvolve` to understand how the dual-loop mechanism distributes search effort across operators over a run.

Operator lifecycle. Figure 5 shows operator selection frequency on two representative tasks. Seed operators dominate the first ∼25 steps while

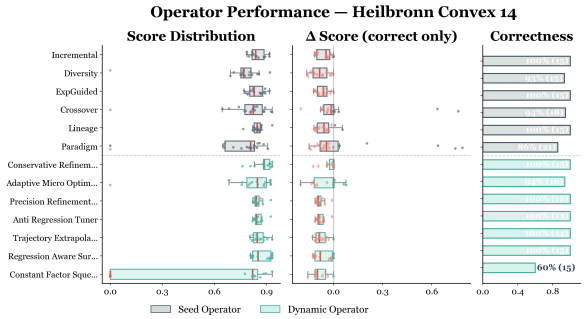


Figure 6: **Per-operator performance** on heilbronn_convex_14. Left: score distribution (red dots = incorrect). Center: Δ Score over parent for correct programs only. Right: success rate with sample count. Dynamic operators (teal) achieve tighter, higher-scoring distributions than most seed operators (gray).

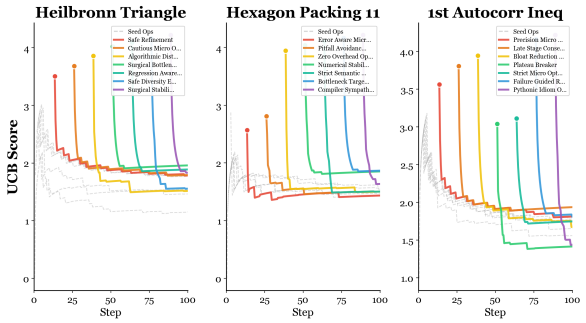


Figure 7: **UCB1 score evolution** on three tasks. Seed operators (gray dashed) converge to a stable band; dynamic operators (colored, birth marked by dot) start with high exploration bonus and stratify gradually.

L1 evidence accumulates; thereafter, the outer loop periodically introduces dynamic operators. UCB1 then reallocates selections toward useful operators and phases out weaker ones. This produces gradual replacement on *circle_packing_rect* and more concentrated late-stage selection on *hexagon_packing_11*, reflecting task-specific operator discovery.

Operator quality. Figure 6 shows per-operator statistics on *heilbronn_convex_14*: score distribution, improvement over parent, and success rate. While exploratory seed operators have wider variance and higher failure rates, dynamic operators such as *ConservativeRefinement* (mean score 0.88, 100% correctness) and *RegressionAwareSurgicalRefiner* (0.87, 100%) form tighter, higher-scoring distributions. Their positive Δ Score further indicates that the outer loop learns exploitation-oriented operators that complement the seed set.

UCB dynamics. Figure 7 traces the UCB1 score of each operator over the run on three tasks. New

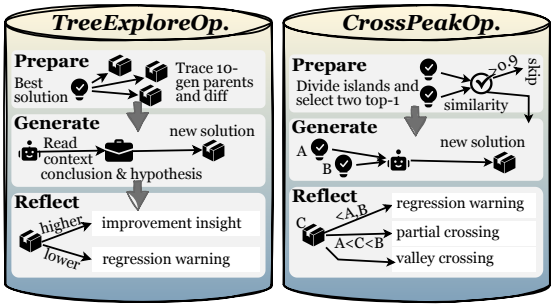


Figure 8: **Case study of synthesized discovery operators:** *TreeExploreOp* and *CrossPeakOp*.

dynamic operators initially receive a high score from the exploration bonus in Equation (10), then quickly move toward their empirical reward after a few selections. Across tasks, seed operators converge to a stable band, while dynamic operators stratify by task-specific usefulness. The changing rankings indicate that the outer loop adapts operator synthesis to each problem rather than producing generic reusable prompts.

4.7 Case Study

Figure 8 illustrates two operators from the outer loop. Both follow the same discovery-operator interface, but implement different methodological moves: *TreeExploreOp* performs lineage-based local exploration, whereas *CrossPeakOp* combines high-performing candidates from separated search regions. The key point is that `MetaEvolve` evolves complete epistemic actions, including what history to inspect, how to prompt the LLM, and what typed experience to write back after evaluation; additional synthesized operators appear in Section D.

5 Conclusion

We presented `MetaEvolve`, a framework for LLM-driven discovery that evolves not only candidate solutions, but also the executable discovery operators that decide what evidence to read, how to intervene, and what experience to retain. Across mathematical optimization tasks, `MetaEvolve` achieves strong open-framework performance and shows clear gains on problems requiring structured methodological shifts. Ablations further indicate that operator evolution and higher-level epistemic memory contribute complementary benefits beyond a fixed evolutionary loop. More broadly, our results suggest that automated scientific discovery systems should be evaluated not only by the answers they find, but by their capacity to refine the very methods through which and how discovery is conducted.

Limitations

This work evaluates `MetaEvolve` on a focused suite of optimization tasks, while ongoing and future experiments can further extend the study to broader scientific, algorithmic, and systems discovery settings. The framework is also naturally compatible with stronger backbone LLMs, which may improve both candidate generation and the synthesis of higher-quality discovery operators.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, and 1 others. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.

Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. 2016. Learning to learn by gradient descent by gradient descent. In *Advances in Neural Information Processing Systems*, volume 29, pages 3981–3989.

Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. 2025. Codeevolve: An open source evolutionary coding agent for algorithm discovery and optimization. *arXiv preprint arXiv:2510.14150*.

Francis Bacon. 1620. *Novum Organum*. John Bill.

Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. 2026. Adaevolve: Adaptive llm driven zeroth-order optimization. *Preprint*, arXiv:2602.20133.

Tianlong Chen, Xiaohan Chen, Wuyang Chen, Howard Heaton, Jialin Liu, Zhangyang Wang, and Wotao Yin. 2022. Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research*, 23(189):1–59.

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and 1 others. 2023. Symbolic discovery of optimization algorithms. *Advances in neural information processing systems*, 36:49205–49233.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*.

Álvaro Fialho. 2010. *Adaptive operator selection for optimization*. Ph.D. thesis, Université Paris Sud-Paris XI.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2023. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers. *arXiv e-prints*, pages arXiv–2309.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, and 1 others. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2025. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*.

Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z. Pan, Alexander Du, Kurt Keutzer, Alvin Cheung, Alexandros G. Dimakis, Koushik Sen, Matei Zaharia, and Ion Stoica. 2026. Evox: Meta-evolution for automated discovery. *Preprint*, arXiv:2602.23413.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594.

Luke Metz, Niru Maheswaranathan, Jeremy Nixon, Daniel Freeman, and Jascha Sohl-Dickstein. 2019. Understanding and correcting pathologies in the training of learned optimizers. In *International Conference on Machine Learning*, volume 97, pages 4556–4565. PMLR.

Jean-Baptiste Mouret and Jeff Clune. 2015. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*.

Alexander Novikov, Ngân Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, and 1 others. 2025a. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, and 1 others. 2025b. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.

Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems.

Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. 2026. Coral: Towards autonomous multi-agent evolution for open-ended discovery. *Preprint*, arXiv:2604.01658.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024a. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, and 1 others. 2024b. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475.

Asankhaya Sharma. 2025. Openevolve: an open-source evolutionary coding agent.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning, 2023. *URL https://arxiv.org/abs/2303.11366*, 8.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*.

Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Zhankui He, Noveen Sachdeva, Isabella Ye, Weili Wang, Chi Wang, and 1 others. 2026. Pacevolve: Enabling long-horizon progress-aware consistent evolution. *arXiv preprint arXiv:2601.10657*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, and 1 others. 2025. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*.

A Seed Operators

`MetaEvolve` initializes the operator population $\mathbf{O}_0$ with six hand-designed seed operators spanning the exploitation–exploration spectrum. Each operator follows the $(\mathbf{p}_o, \mathbf{g}_o, \mathbf{r}_o)$ protocol of Equation (3), differing in which memory layers it reads, how it selects and presents parents, and what structured experience it writes back.

- **IncrementalMutation** (exploitation). **Prepare:** selects the best correct program as parent and retrieves recent `improvement_insight` and `error_pattern` experiences from L1 memory. Constructs a prompt addon that enumerates prior successful small changes and known failure modes, and overrides the patch type to `diff`. **Generate:** appends an explicit instruction to identify a single high-impact modification and produce only the minimal diff required. **Reflect:** annotates successful improvements with a note that diff-based exploitation in the current region remains productive.
- **DiversityMutation** (exploration). **Prepare:** samples a parent from the least-populated island to maximize structural diversity, and retrieves `error_pattern` and `stagnation_note` experiences. Overrides the patch type to `full` and raises the sampling temperature to 1.0. **Generate:** instructs the LLM to produce a completely new implementation without preserving the existing program structure, explicitly requesting an alternative algorithmic paradigm. **Reflect:** if the full rewrite improves the score, records that diversity-driven exploration from an under-explored island succeeded.
- **ExperienceGuided** (feedback synthesis). **Prepare:** selects the best correct program and queries all four L1 experience categories: `improvement_insight`, `regression_warning`, `error_pattern`, and `stagnation_note`. Compiles them into a structured guidance document organized by what worked, what to avoid, what errors to fix, and which directions stagnated. **Generate:** prepends the guidance to the system message and instructs the LLM to prioritize the positive patterns, actively avoid the regressive ones, and address the listed errors. **Reflect:** tags the resulting experience with the operator identity to track which experience combinations prove most informative.
- **CrossoverFusion** (recombination). **Prepare:** samples a parent from one island and retrieves archive and top-k inspirations from other islands, retrieving `improvement_insight` experiences to identify which strengths each parent carries. If inspiration programs are available, sets the patch type to `cross`; otherwise falls back to `diff`. **Generate:** presents both the parent and the highest-scoring inspiration side by side, reporting their respective scores, and instructs the LLM to iden-

tify specific strengths of each program and merge them into a single superior solution. **Reflect:** inherits the default reflection, recording whether the crossover surpassed both donor programs.

- **LineageAnalysis** (trajectory exploitation). **Prepare:** retrieves the ancestry chain of the current best program (up to five ancestors) and formats a score-over-generations table showing the progression of improvements. Also retrieves recent `improvement_insight` experiences. Overrides the patch type to `diff`. **Generate:** instructs the LLM to study the score trajectory, identify the pattern of changes that produced successive improvements, and determine what incremental step logically follows. **Reflect:** annotates successful improvements as lineage-guided continuations, reinforcing the ancestry-tracing strategy in future operator selection.
- **ParadigmShift** (stagnation escape). **Prepare:** monitors the most recent programs and raises an `OperatorSkip` if any recent improvement is detected, activating only when the search has genuinely stagnated. Deliberately selects a *below-median* correct program as parent rather than the best, retrieves `error_pattern` and `stagnation_note` experiences to enumerate exhausted directions, sets the patch type to `full`, and raises the temperature to 1.2. **Generate:** instructs the LLM to abandon the current approach entirely, suggesting alternative mathematical formulations, novel data structures, unconventional optimization strategies, or techniques borrowed from adjacent domains. **Reflect:** if the radical rewrite breaks through the plateau, records a paradigm-shift success and recommends further exploration in the newly discovered direction.

Together, these six operators span local exploitation (IncrementalMutation, LineageAnalysis), guided recombination (CrossoverFusion, ExperienceGuided), and broad exploration (DiversityMutation, ParadigmShift). The outer loop's UCB1 bandit adaptively allocates evaluations among them and, as the run progresses, synthesizes new operators that combine or extend the seed strategies (see Section D for examples of such emergent operators). Algorithms 2–7 give pseudocode for each seed operator.

Algorithm 2: IncrementalMutation

Phase Prepare$(\mathbf{M}_t)$
  $x_p \leftarrow \text{BestCorrect}(\mathbf{M}_t)$;
  $\mathcal{I} \leftarrow \text{GetExp}(\mathbf{M}_t, \texttt{improvement_insight}, 3)$;
  $\mathcal{E} \leftarrow \text{GetExp}(\mathbf{M}_t, \texttt{error_pattern}, 2)$;
  addon $\leftarrow$ "Focus on one targeted change" $\oplus\, \mathcal{I} \,\oplus\, \mathcal{E}$;
  return $c_t = (x_p,\ \textit{addon},\ \texttt{patch=diff})$;
Phase Generate$(c_t, \mathbf{L})$
  $(s, u) \leftarrow \text{DefaultPrompt}(c_t)$;
  $s \leftarrow s \oplus$ "Identify ONE change; show minimal diff";
  return $x_{t+1} \leftarrow \mathbf{L}(s, u)$;
Phase Reflect(x_{t+1}, y_{t+1})
  $z \leftarrow \text{RuleClassify}(\Delta s)$;
  if $z.\textit{type} = \texttt{improvement_insight}$ **then**
    $z.\textit{content} \oplus=$ "diff exploitation effective";
  return z;

Algorithm 3: DiversityMutation

Phase Prepare$(\mathbf{M}_t)$
  pops $\leftarrow \text{IslandPopulations}(\mathbf{M}_t)$;
  $i^* \leftarrow \arg\min_i$ *pops*$[i]$;
  $x_p \leftarrow \text{RandomFrom}(\text{Island}_{i^*})$;
  $\mathcal{E} \leftarrow \text{GetExp}(\texttt{error_pattern}, 3) \cup \text{GetExp}(\texttt{stagnation_note}, 2)$;
  addon $\leftarrow$ "Try a fundamentally different approach" $\oplus\, \mathcal{E}$;
  return $c_t = (x_p,\ \textit{addon},\ \texttt{patch=full},\ T = 1.0)$;
Phase Generate$(c_t, \mathbf{L})$
  $(s, u) \leftarrow \text{DefaultPrompt}(c_t)$;
  $s \leftarrow s \oplus$ "EXPLORATION mode: write a completely new implementation";
  return $x_{t+1} \leftarrow \mathbf{L}(s, u)$;
Phase Reflect(x_{t+1}, y_{t+1})
  $z \leftarrow \text{RuleClassify}(\Delta s)$;
  if $z.\textit{type} = \texttt{improvement_insight}$ **then**
    $z.\textit{content} \oplus=$ "diversity exploration from island i^* succeeded";
  return z;

B Task Descriptions

All eight tasks are drawn from the AlphaEvolve mathematical optimization benchmark (Novikov et al., 2025a). Each task requires the system to evolve a Python program that constructs a mathematical object (a configuration, a function, or a point set) optimizing a specific metric.

- **Circle Packing Rect** (↑). Pack n non-overlapping circles into the plane so that the sum of their radii is maximized while the minimum circumscribing axis-aligned rectangle has unit area.
- **Heilbronn Convex-13 / 14** (↑). Place $n = 13$ (or 14) points in the unit square so as to maximize the minimum area of any triangle formed by three points that lie on the convex hull.
- **Hexagon Packing-11 / 12** (↑). Pack $n = 11$ (or

Algorithm 4: EXPERIENCEGUIDED

Phase Prepare$(\mathbf{M}_t)$
- $x_p \leftarrow \text{BestCorrect}(\mathbf{M}_t)$;
- $\mathcal{I} \leftarrow \text{GetExp}(\texttt{improvement_insight}, 3)$;
- $\mathcal{R} \leftarrow \text{GetExp}(\texttt{regression_warning}, 2)$;
- $\mathcal{E} \leftarrow \text{GetExp}(\texttt{error_pattern}, 3)$;
- $\mathcal{S} \leftarrow \text{GetExp}(\texttt{stagnation_note}, 2)$;
- *addon* $\leftarrow \text{BuildGuidance}(\mathcal{I}, \mathcal{R}, \mathcal{E}, \mathcal{S})$; `// structured by category`
- **return** $c_t = (x_p,\ \textit{addon},\ \texttt{patch=diff})$;

Phase Generate$(c_t, \mathbf{L})$
- $(s, u) \leftarrow \text{DefaultPrompt}(c_t)$;
- $s \leftarrow s \oplus$ "Prioritize positive patterns; avoid regressions; fix errors";
- **return** $x_{t+1} \leftarrow \mathbf{L}(s, u)$;

Phase Reflect(x_{t+1}, y_{t+1})
- $z \leftarrow \text{RuleClassify}(\Delta s)$;
- $z.\textit{content} \leftarrow$ "[ExperienceGuided] " $\oplus\ z.\textit{content}$;
- **return** z;

Algorithm 5: CROSSOVERFUSION

Phase Prepare$(\mathbf{M}_t)$
- $(x_p, \mathcal{A}, \mathcal{K}) \leftarrow \text{Sample}(\mathbf{M}_t)$; `// parent + cross-island inspirations`
- $\mathcal{I} \leftarrow \text{GetExp}(\texttt{improvement_insight}, 3)$;
- **if** $\mathcal{A} \cup \mathcal{K} \neq \varnothing$ **then**
  - *patch* $\leftarrow$ `cross`;
- **else**
  - *patch* $\leftarrow$ `diff`;
- *addon* $\leftarrow$ "Combine strengths of parent and inspirations" $\oplus\ \mathcal{I}$;
- **return** $c_t = (x_p,\ \mathcal{A},\ \mathcal{K},\ \textit{addon},\ \textit{patch})$;

Phase Generate$(c_t, \mathbf{L})$
- $(s, u) \leftarrow \text{DefaultPrompt}(c_t)$;
- $x_{\text{insp}} \leftarrow \text{TopInspiration}(\mathcal{K}, \mathcal{A})$;
- $s \leftarrow s \oplus$ "CROSSOVER: parent score $s(x_p)$, inspiration score $s(x_{\text{insp}})$. Merge strengths.";
- **return** $x_{t+1} \leftarrow \mathbf{L}(s, u)$;

Phase Reflect(x_{t+1}, y_{t+1})
- **return** RuleClassify(Δs); `// default reflection`

Algorithm 6: LINEAGEANALYSIS

Phase Prepare$(\mathbf{M}_t)$
- $x^* \leftarrow \text{BestCorrect}(\mathbf{M}_t)$;
- $\mathcal{H} \leftarrow \text{GetAncestry}(x^*,\ \text{depth} = 5)$;
- *table* $\leftarrow [(g_i, s_i) \mid x_i \in \mathcal{H}]$; `// score-over-generations`
- $\mathcal{I} \leftarrow \text{GetExp}(\texttt{improvement_insight}, 3)$;
- *addon* $\leftarrow$ *table* $\oplus$ "Continue the trajectory of successful changes" $\oplus\ \mathcal{I}$;
- **return** $c_t = (x^*,\ \textit{addon},\ \texttt{patch=diff})$;

Phase Generate$(c_t, \mathbf{L})$
- $(s, u) \leftarrow \text{DefaultPrompt}(c_t)$;
- $s \leftarrow s \oplus$ "LINEAGE mode: study the score progression; what step comes next?";
- **return** $x_{t+1} \leftarrow \mathbf{L}(s, u)$;

Phase Reflect(x_{t+1}, y_{t+1})
- $z \leftarrow \text{RuleClassify}(\Delta s)$;
- **if** $z.\textit{type} = \texttt{improvement_insight}$ **then**
  - $z.\textit{content} \oplus=$ "lineage-guided continuation effective";
- **return** z;

Algorithm 7: PARADIGMSHIFT

Phase Prepare$(\mathbf{M}_t)$
- $\mathcal{W} \leftarrow \text{Recent}(\mathbf{M}_t, W)$; `// last` W `programs`
- **if** $\exists\, x \in \mathcal{W} :\ s(x) > s(x^*)$ **then**
  - **raise** OPERATORSKIP("no stagnation");
- $\mathcal{C} \leftarrow \{x \in \mathbf{M}_t \mid x \text{ correct}\}$;
- $x_p \leftarrow \text{RandomFrom}(\text{BottomHalf}(\mathcal{C}))$; `// deliberately weak parent`
- $\mathcal{E} \leftarrow \text{GetExp}(\texttt{error_pattern}, 3) \cup \text{GetExp}(\texttt{stagnation_note}, 3)$;
- *addon* $\leftarrow$ "Search STAGNATED. Try a completely different paradigm" $\oplus\ \mathcal{E}$;
- **return** $c_t = (x_p,\ \textit{addon},\ \texttt{patch=full},\ T = 1.2)$;

Phase Generate$(c_t, \mathbf{L})$
- $(s, u) \leftarrow \text{DefaultPrompt}(c_t)$;
- $s \leftarrow s \oplus$ "PARADIGM SHIFT: abandon current approach. Try novel formulations, data structures, or cross-domain strategies.";
- **return** $x_{t+1} \leftarrow \mathbf{L}(s, u)$;

Phase Reflect(x_{t+1}, y_{t+1})
- $z \leftarrow \text{RuleClassify}(\Delta s)$;
- **if** $z.\textit{type} = \texttt{improvement_insight}$ **then**
  - $z.\textit{content} \oplus=$ "paradigm shift broke through stagnation";
- **return** z;

12) non-overlapping unit regular hexagons inside a larger regular hexagon, minimizing the side length of the outer hexagon. The metric is the inverse side length (↑).

- **Erdős Minimum Overlap** (↓). Construct a function $f : [0, 1] \to \{0, 1\}$ with $\int f = 1/2$ that minimizes the C_5 overlap integral, an upper bound in Erdős's minimum overlap problem.
- **First Autocorrelation Inequality** (↓). Construct a non-negative function f on $[0, 1/2]$ that minimizes the C_1 ratio between the first autocorrelation moment and a normalization term, tightening a classical inequality.
- **Third Autocorrelation Inequality** (↓). Same family as above but for the third-order autocorrelation constant C_3.

C More Results

Figure 9 provides an additional qualitative comparison on *first_autocorr_ineq*. While all methods search over executable candidate programs, the resulting solutions differ in how they shape the autocorrelation profile. The visualization complements the quantitative result in Table 1, where

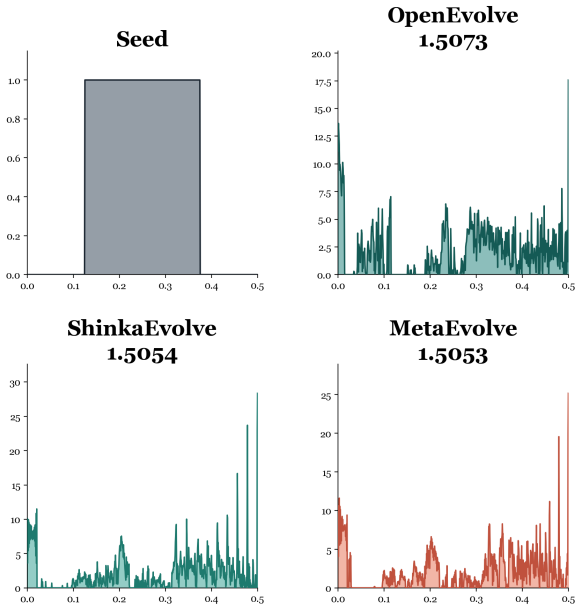


Figure 9: Visualization of the solutions discovered by OpenEvolve, ShinkaEvolve and MetaEvolve on first_autocorr_ineq problem.

MetaEvolve reaches the strongest reported score on this task.

D Case Study

Figures 10 to 15 show six discovery operators synthesized by MetaEvolve. They share the same prepare–generate–reflect interface, but instantiate distinct methodological structures: lineage extrapolation, function-level transfer, multi-source synthesis, cross-peak exploration, ancestral patch reuse, and experience-driven leaps. These examples illustrate that meta-evolution does not merely tune a prompt or choose among fixed mutations; it invents new operator programs that decide what evidence to read, what transformation to request from the LLM, and what typed experience to write back.

E Prompt Templates

We present three representative prompts that drive the core loops of MetaEvolve: operator synthesis (outer loop), experience reflection (L1 memory writing), and meta-summarization (L3 abstraction). Together they illustrate how the system reads evidence, creates new methods, and distills methodological knowledge.

Prompt 1: Operator Synthesis (Outer Loop). This prompt asks the LLM to design a new discovery operator as executable code. The system message specifies the operator protocol and available APIs; the user message provides the current search state.

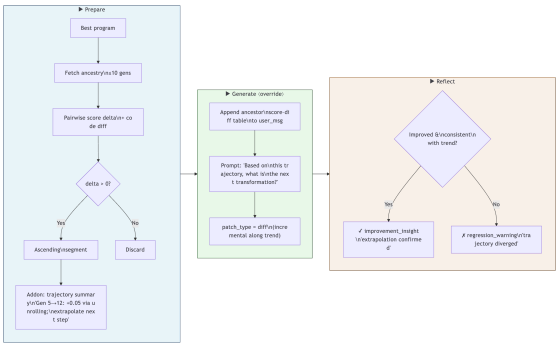


Figure 10: **TrajectoryExtrapolator.** The operator traces the ancestry of the current best program, extracts score-improving code diffs, and asks the LLM to extrapolate the next logical transformation along the observed trajectory.

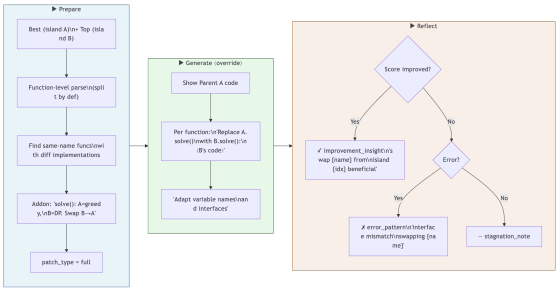


Figure 11: **ScopedFunctionSwap.** The operator compares top programs from different islands at function granularity, swaps a selected implementation into the parent codebase, and reflects on interface mismatches or beneficial transfers.

```
[System Message]
You are an expert in evolutionary algorithm design. Your task is to create a new evolutionary operator for code optimization.

An operator has three phases:
1. prepare() - select parents and build context (what the LLM sees)
2. generate() - optionally customize the LLM prompt (how code is generated)
3. reflect() - optionally customize the analysis of outcomes (what was learned)

The operator does NOT call the LLM itself - it returns data structures that the framework uses.

## Available APIs
{db_api} // ProgramDatabase method signatures

## Rules
1. prepare() MUST return an OperatorContext. Use db.sample() as fallback.
2. prepare() should read relevant experiences
```

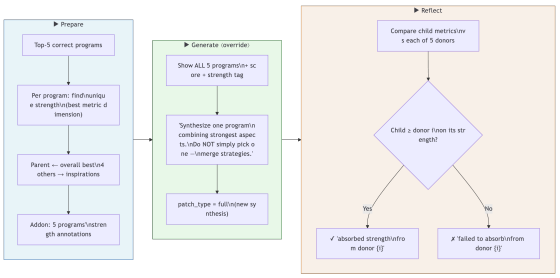


Figure 12: **EnsembleSynthesis.** The operator retrieves multiple high-performing correct programs, annotates their distinct metric-level strengths, and asks the LLM to synthesize a single solution that combines them.

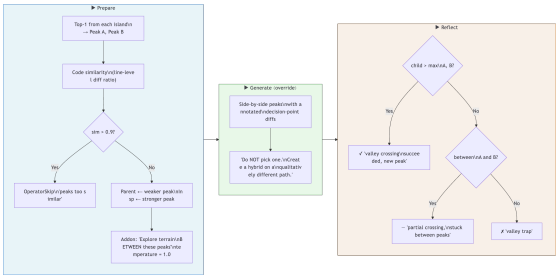


Figure 13: **CrossPeakExplorer.** The operator selects top programs from separated islands, avoids near-duplicate peaks, and explores a hybrid path between qualitatively different solution strategies.

```
via db.get_experiences_by_type().
3. generate() and reflect() are optional —
only override if needed.
. . .
7. Keep the logic focused — one clear strategy,
not a kitchen sink.
8. The system_prompt_addon should be concise
and actionable.
. . .
10. If your strategy doesn't apply, raise
OperatorSkip("reason").
[User Message]
## Current Operator Stats
{operator_stats} // per-operator attempts,
success rate, mean reward
## Recent Experiences (last 15)
{recent_experiences} // typed L1 experiences
with score deltas
## Recent Score Progression (last 20
generations)
{score_progression}
## Successful Transformations (top 5
improvements)
{transformations}
Based on the search trajectory above, design
a NEW operator that addresses a gap in the
current operator portfolio. Think about what
strategy is missing or underperforming.
```

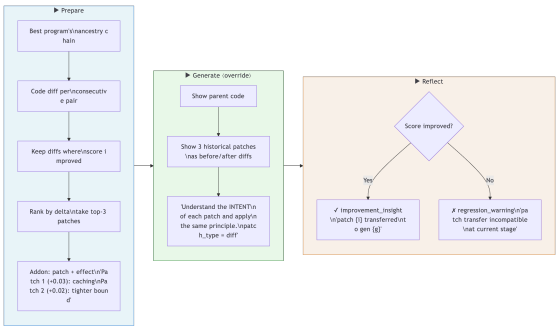


Figure 14: **AncestralPatchTransfer.** The operator mines historically successful ancestor diffs and asks the LLM to transfer the underlying patch intent to the current best program.

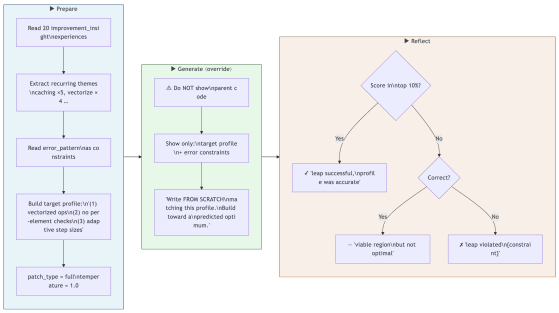


Figure 15: **LatentOutcomeLeapOperator.** The operator aggregates repeated improvement insights and error patterns into a target profile, then prompts a from-scratch solution aimed at the inferred optimum.

```
Output ONLY a Python class. You MUST
implement prepare(). Override generate()
and/or reflect() only if needed.
```

Prompt 2: Experience Reflection (L1 Memory Writing). After each evaluation, the operator's `reflect()` method classifies the outcome into one of four experience types and generates a structured textual reflection. The default implementation is rule-based; an optional LLM enrichment step can add actionable insights.

```
[Rule-Based Classification]
if program failed correctness check:
type ← error_pattern
content ← "Program failed (score={s}).
Operator {op} produced
incorrect output. Error: {detail}"
elif Δscore > 0:
type ← improvement_insight
content ← "Score improved by {Δ}
({parent}→{child}).
Operator {op} found a beneficial change."
elif Δscore < 0:
```

```
type ← regression_warning
content ← "Score regressed by {Δ}. Operator
{op}'s change
was counterproductive."
else:
type ← stagnation_note
content ← "No score change. Operator {op}
produced a neutral
mutation."
```

[Optional LLM Enrichment — System Message]

```
You are analyzing the outcome of an
evolutionary code optimization step. Enrich
the following reflection with a brief,
actionable insight (1–2 sentences).
```

[User Message]

```
Operator: {operator_name}
Outcome: {experience_type}
Score delta: {delta}
Operator's reflection: {reflection}

Add one actionable insight for future
iterations. Be specific and concise.
```

Prompt 3: Meta-Summarization (L3 Abstraction). The meta-summarizer periodically distills L1 experiences and L0 solution trajectories into high-level methodological insights (L3 memory). This is a three-step pipeline; we show Step 2 (global insight extraction), which is the core analytical step.

[System Message]

```
You are an expert programming assistant
analyzing specific program evaluation results
to extract actionable optimization insights.
Focus on concrete performance data and
implementation details from the actual programs
that were evaluated.
```

[User Message]

```
## Individual Program Summaries
{individual_summaries} // from Step 1

## Previous Global Insights (if any)
{previous_insights} // accumulated L3 memory

## Current Best Program
{best_program_info}

Analyze the SPECIFIC program evaluation
results above to extract concrete optimization
insights. Reference specific programs and
their results. Pay special attention to the
current best program.

Update or create insights in these sections:

## Successful Algorithmic Patterns
- Identify implementation changes that led to
score improvements
- Highlight patterns from the current best
program (2-4 points)

## Ineffective Approaches
- Identify changes that worsened performance
(2-4 points)

## Implementation Insights
- Analyze what makes the current best program
effective
- Connect implementation details to performance
impact (2-4 points)

## Performance Analysis
- Compare programs' performance against the
current best
- Identify score patterns and correlations (2-4
points)

Base insights on the ACTUAL program summaries
and previous insights. Do not make
recommendations — ONLY PERFORM THE ANALYSIS.
```

Prompt 1 feeds the outer loop (Section 3.3): its output is a new executable operator that is validated, registered, and immediately available for UCB1 selection. Prompt 2 populates L1 memory after every evaluation, providing the typed experiences that both seed and dynamic operators read during their `prepare()` phase. Prompt 3 runs periodically to compress L1 evidence into L3 meta-experiences, which in turn inform the next invocation of Prompt 1 via the `{recent_experiences}` field. The three prompts thus form a closed epistemic cycle: reflect → synthesize → abstract → reflect.

F Best Discovered Programs

We present the three programs discovered by `MetaEvolve` that surpass the AlphaEvolve benchmark. All were found by the Gemini-3.1-Pro backbone within a 100-evaluation budget.

Hexagon Packing-12 (score $= 0.2537$, AlphaEvolve $= 0.2537$; ↑)

The program packs 12 unit regular hexagons inside a larger regular hexagon with outer side length 3.9417 (inverse $= 0.25371$), improving upon AlphaEvolve's 3.9419 (0.25368). It uses a multi-stage pipeline: (1) discrete topology search via 20,000-chain parallel simulated annealing on a hexagonal grid, (2) continuous relaxation with the Adam optimizer, (3) exact LP with cutting planes, and (4) angle refinement via bounded scalar minimization.

```python
import numpy as np
from scipy.optimize import
    minimize_scalar, linprog

def hexagon_packing_12():
    SQRT3 = 1.7320508100147274
    # 1. Precompute hexagonal grid (61
        candidate positions)
    grid = []
    for q in range(-4, 5):
        for r in range(-4, 5):
            if -4 <= -q-r <= 4:
                grid.append((q, r))
    grid = np.array(grid)
```

```
    pts = np.column_stack((1.5 * grid
        [:, 0],
        SQRT3 * (grid[:, 1] + grid[:,
            0] * 0.5)))

    # 2. Vectorized parallel SA (20000
        chains, 350 steps)
    #    - Neighborhood-biased swap
        mutations on grid
    #    - Projection-based width
        evaluation across 60 angles
    #    - Boltzmann acceptance with
        exponential cooling

    # 3. Adam continuous relaxation
        (800 steps)
    #    - Smooth-max hex containment
        via logsumexp
    #    - Pairwise overlap penalty
        with increasing lambda
    #    - Joint optimization of
        positions and outer angle

    # 4. LP with cutting planes (top
        100 candidates)
    #    - Linear hex-distance
        separation constraints
    #    - Iterative violation
        detection and constraint
        addition
    #    - Angle refinement via
        minimize_scalar for top 30

    # Returns inner_hex_data (12,3),
        outer_hex_data (3,),
    #         outer_hex_side_length
```

Erdős Minimum Overlap (score $= 0.38090$, AlphaEvolve $= 0.38092$; $\downarrow$)

The program constructs a step function $h : [0, 2] \to \mathbb{R}$ with $\int h = 1$ that achieves $C_5 = 0.38090$, improving upon AlphaEvolve's 0.38092. It uses JAX with 8,192 parallel optimization runs, FFT-based correlation evaluation, sigmoid projection for integral constraints, and logsumexp smooth-max with temperature annealing. An evolutionary outer loop selects the top 1/8 candidates each epoch and generates mutants with scaled noise.

```
import jax, jax.numpy as jnp, optax,
    numpy as np

class ErdosOptimizer:
    # N=200 intervals, 20000 Adam steps
        , batch=8192
    def run_optimization(self):
        # Sigmoid projection with
            Newton's method
        #   ensures integral constraint
             sum(h)*dx = 1
        def project(latent):
            c, _ = jax.lax.scan(
                newton_step, 0.0, None,
                 length=20)
            return jax.nn.sigmoid(
                latent + c), c

        # FFT-based overlap evaluation
        #   h_fft = rfft(h, 2N); corr =
             irfft(h_fft*W - |h_fft|^2)
        def eval_corr(h): ...

        # Loss = logsumexp(corr/temp)*
            temp + regularizers
        #   temp decays from 0.01 to 1e
            -5 over training
        def objective_fn(latent, temp):
             ...

        # Warmup-cosine-decay schedule
            + gradient clipping
        # Evolutionary: top-K selection
             + 8 noise tiers
        #   [0, 0.01, 0.03, 0.07, 0.15,
             0.3, 0.6, 1.0]
        # Returns h with C5 = max(
            correlation)
```

Third Autocorrelation (score $= 1.4551$, AlphaEvolve $= 1.4556$; $\downarrow$)

The program constructs a non-negative function f on $[0, 0.5]$ that achieves $C_3 = 1.4551$, improving upon AlphaEvolve's 1.4556. It uses JAX with 16,384 parallel runs, 400 intervals, and 40,000 optimization steps. The autoconvolution is computed via FFT, and the smooth-max uses logsumexp with exponentially decaying temperature. An evolutionary outer loop applies elite selection, crossover, and frequency-domain noise mutation with decaying scale.

```
import jax, jax.numpy as jnp, optax,
    numpy as np

class C3Optimizer:
    # N=400 intervals, 40000 steps,
        batch=16384
    def run_optimization(self):
        # Frequency-domain
            initialization
        #   random spectral
            coefficients with power-law
             decay
        #   integral normalization: f =
             f / (sum(f)*dx)

        # FFT autoconvolution
            evaluation
        #   rfft_f = rfft(f, 2N); conv
            = irfft(rfft_f^2) * dx
        #   C3 = max(|conv|) / integral
            ^2

        # Loss = logsumexp(|conv| * dx/
            temp) * temp
        #   temp: 0.05 * (0.005^
            progress)
        # Adam + warmup-cosine-decay +
            grad clipping

        # Evolutionary outer loop (
```

```
            every 2000 steps):
        #   Elite selection: top 1/8 by
             loss
        #   Crossover: convex
            combination of two elites
        #   Mutation: frequency-domain
            noise, scale decays
        #     noise_scale = 0.15 * (1 -
             progress)^1.5

        # Returns f_values with C3
            upper bound
```

G Seed Operator Prompts

Each seed operator injects text into the LLM prompt at two points: a `system_prompt_addon` composed during **Prepare** (appended to the base system message), and an additional instruction appended during **Generate**. The **Reflect** phase uses a shared rule-based classifier (see Prompt 2 in Section E); operators that override it append a short annotation. Below we list the exact injected text for each seed operator. Placeholders in braces are filled at runtime.

1. IncrementalMutation

Prepare (system_prompt_addon):

Focus on making small, targeted improvements to the current best solution. Identify the most impactful single change that could improve the score. Do NOT rewrite large sections — change only what is necessary.

Recent successful improvements:
{improvement_insight entries, up to 3}
Errors to avoid:
{error_pattern entries, up to 2}

Generate (appended to sys_msg):

You are in INCREMENTAL mode. Identify ONE specific change that will improve the score. Show only the minimal diff needed.

Reflect (annotation on improvement):

Small diff-based mutation was effective — continue exploiting this region.

2. DiversityMutation

Prepare (system_prompt_addon):

Try a FUNDAMENTALLY DIFFERENT approach from the current solution. Do not make incremental changes — rethink the core algorithm or strategy.

Approaches that FAILED (do not repeat these mistakes):
{error_pattern entries, up to 3}
Stagnant directions (these led nowhere):
{stagnation_note entries, up to 2}

Generate (appended to sys_msg):

You are in EXPLORATION mode. The goal is diversity, not refinement. Write a completely new implementation. Do NOT preserve the existing structure.

Reflect (annotation on improvement):

Full rewrite from underexplored island succeeded — diversity-driven exploration found a better approach.

3. ExperienceGuided

Prepare (system_prompt_addon):

Based on analysis of recent evolution outcomes:
What worked (apply these patterns):
{improvement_insight entries, up to 3}
What to avoid (these changes hurt performance):
{regression_warning entries, up to 2}
Common errors to fix (these programs failed validation):
{error_pattern entries, up to 3}
Stagnant directions (no progress):
{stagnation_note entries, up to 2}

Generate (appended to sys_msg):

IMPORTANT: The recommendations above come from analyzing previous evolution outcomes. Prioritize the 'what worked' insights, actively avoid the 'what to avoid' patterns, and fix the listed errors.

Reflect: Tags experience with [ExperienceGuided] prefix.

4. CrossoverFusion

Prepare (system_prompt_addon):

```
Combine the strengths of the parent program with the inspiration programs. The current best solution has combined_score={best_score}. Try to surpass it by merging the best ideas from both programs.

Recent improvements (merge these patterns):
{improvement_insight entries, up to 3}
```

Generate (appended to sys_msg):

```
You are in CROSSOVER mode. The parent has score {parent_score} and the inspiration has score {insp_score}. Identify the specific strengths of each and merge them into a single superior solution.
```

Reflect: Default rule-based classification (no custom annotation).

5. LineageAnalysis

Prepare (system_prompt_addon):

```
The best solution evolved through these generations:
Gen {g1}: score={s1}
Gen {g2}: score={s2}
...
Gen {gN}: score={sN} (current best)

Analyze the pattern of improvements in this lineage. Continue the trajectory of successful changes.

Recent improvements to continue:
{improvement_insight entries, up to 3}
```

Generate (appended to sys_msg):

```
You are in LINEAGE mode. Study the progression of scores above. What pattern of changes produced these improvements? What incremental step would logically come next?
```

Reflect (annotation on improvement):

```
Lineage-guided continuation was effective.
```

6. ParadigmShift

Prepare (activates only when stagnation is detected):

```
The search has STAGNATED — no improvement in recent generations. The current approaches have hit a local optimum. You MUST try a completely different algorithmic paradigm.

Approaches that have FAILED (do not repeat):
{error_pattern entries, up to 3}
Directions that led NOWHERE:
{stagnation_note entries, up to 3}
```

Generate (appended to sys_msg):

```
You are in PARADIGM SHIFT mode. The current approach is exhausted. Consider: different mathematical formulations, alternative data structures, novel optimization strategies, or approaches from related but different domains. Bold, unconventional ideas are strongly preferred over safe incremental changes.
```

Reflect (annotation on improvement):

```
Paradigm shift broke through stagnation! Consider further exploration in this new direction.
```

H Information About Use Of AI Assistants

AI assistance was used only for writing polishing and auxiliary coding support in this work.

I Potential Risks

This work focuses on evaluator-guided discovery in optimization problems, where candidates are executable programs assessed by task-specific objective functions. We do not identify risks beyond those commonly associated with LLM-based code generation and automated experimentation, such as producing incorrect, insecure, or unintended code when deployed outside controlled evaluators. Accordingly, `MetaEvolve` should be used with standard safeguards for LLM-generated code, including sandboxed execution, evaluator validation, resource limits, and human oversight in high-stakes applications.